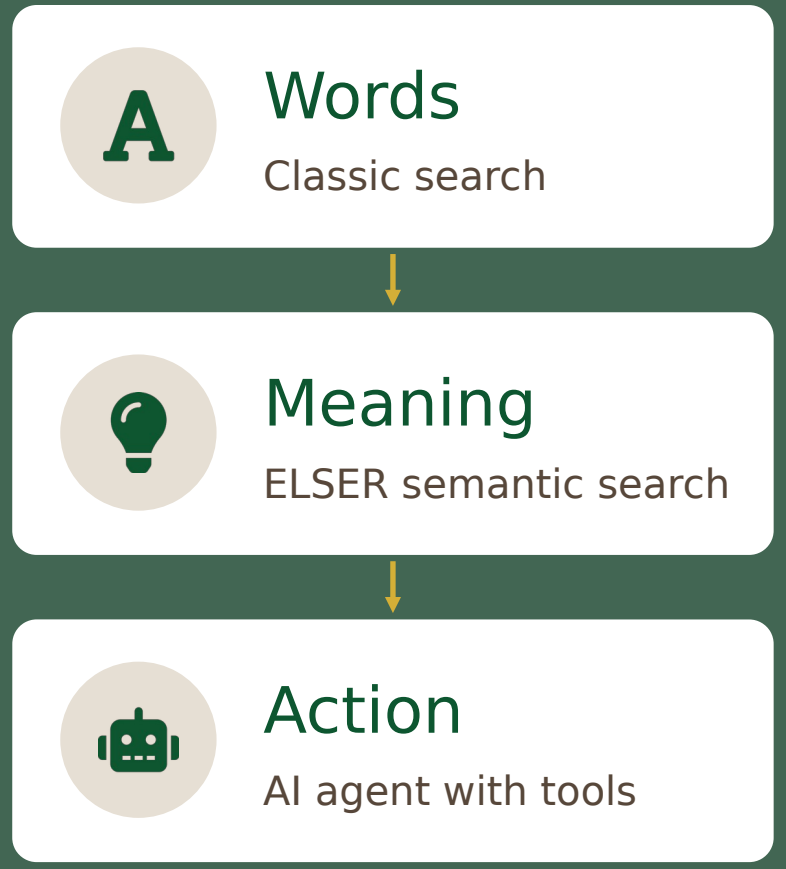
ELASTICSEARCH • ELSER • AGENT BUILDER

From Search to Agents

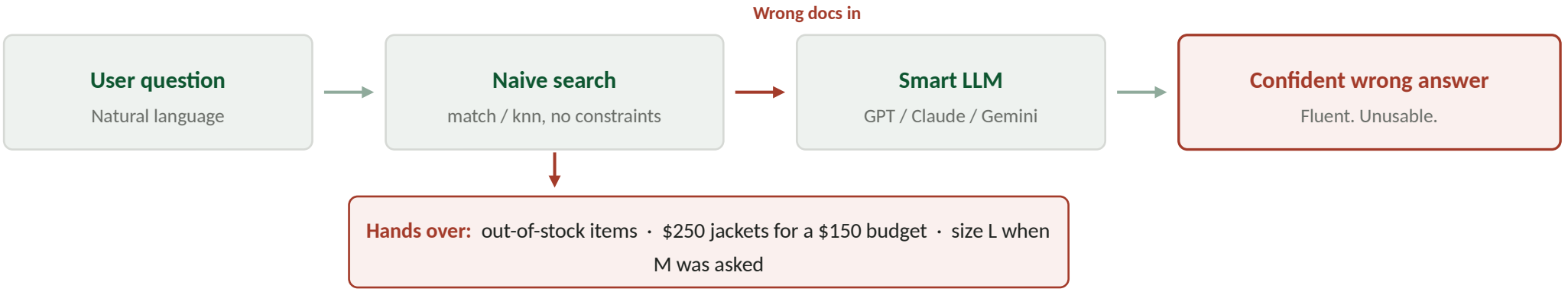
How search grows from matching words, to understanding meaning, to taking action.

Sandheep | SquareShift



The core problem: why AI demos fall apart

The model is fine. The data retrieval is the problem.



The LLM has no memory of your warehouse

No foundation model knows your live catalogue of 15,000 SKUs, or whether size M is on the shelf right now.

Garbage in, fluent garbage out

Hand the model out-of-stock items and it will write a confident, well-structured, completely wrong recommendation.

Prompting cannot fix bad data

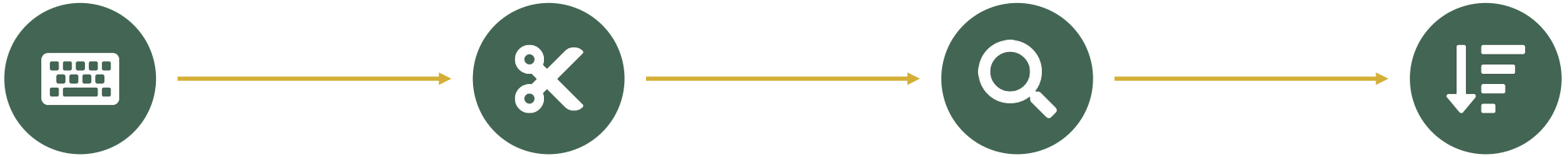
RAG is a retrieval engineering problem, not a prompt engineering problem. No system prompt rescues the wrong five documents.

90% of the time the LLM did exactly what it was told — the failure happened before the model was ever called.

STAGE 1 • CLASSIC SEARCH

Classic search matches words

Fast, predictable and still the right answer for many searches.



You type
a few words

Split
into individual words

Look up
documents with those words

Rank
best word-match first

Works great when…

- Shoppers know the exact product words
- Product names, brands, SKUs
- Speed and explainability matter

Struggles when…

- Shoppers describe a need instead of a name
- Different words mean the same thing
- Words overlap but intent doesn't: “hiking jacket” can return hiking pants

STAGE 2 • SEMANTIC SEARCH

ELSER searches by meaning, not just words

Elastic Learned Sparse Encoder: Elastic's own retrieval model, built into Elasticsearch.

QUERY *“comfortable outdoor shell for wet weather”*

↓

ELSER expands it into weighted, related terms

rain 0.95 **waterproof** 0.88 **jacket** 0.80

shell 0.74 **storm** 0.55 **breathable** 0.42

hiking 0.35

↓

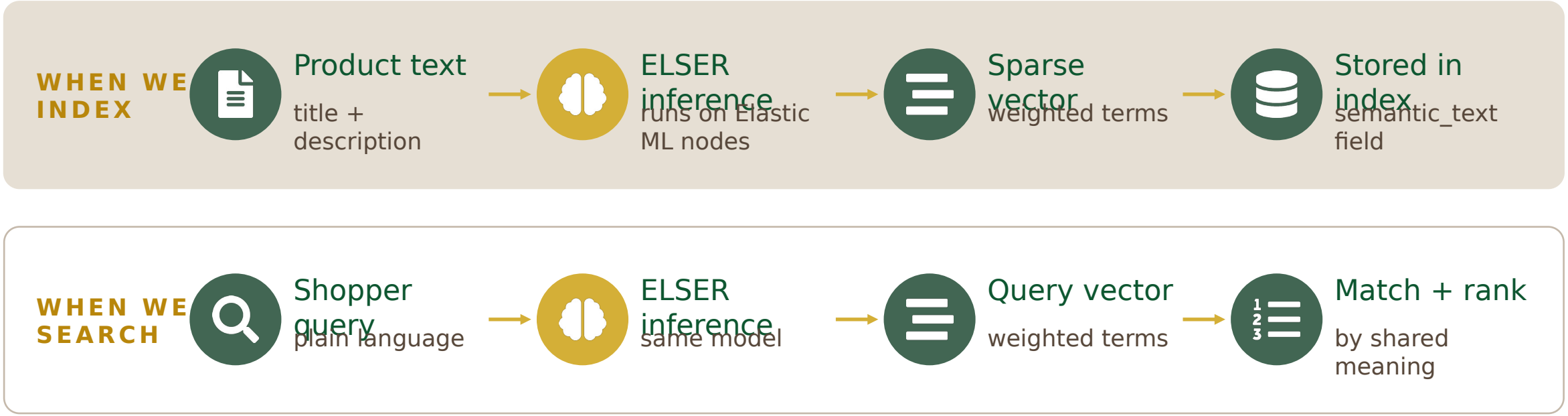
FINDS Rain shells “built for sustained rain”, even with no shared words

Why it's powerful

- Understands synonyms and paraphrases
- Runs inside Elasticsearch, no external embedding service
- Works out of the box, no fine-tuning needed
- Explainable: you can see the terms it matched on

How ELSER works: inference creates the vectors

The same model runs on documents and on queries, so both speak the same language.



Inference = running the ML model inside Elasticsearch to turn text into a vector. A sparse vector keeps only the few terms that matter, each with a weight, so matching stays fast and easy to explain.

THE GAP

Relevant isn't the same as right

Semantic search finds the right kind of product. It doesn't enforce rules or compare.

SHOPPER ASKS *“Waterproof jackets under $150, size M, in stock. Compare the top two.”*



Waterproof jacket

Meaning

ELSER handles it



Under $150

Exact price rule

Not guaranteed



Size M

Exact attribute

Not guaranteed



In stock

Exact stock check

Not guaranteed



Compare top two

A second step

Not guaranteed

Hard rules need filters and logic, not just ranking. That's the job of an AI agent with the right tools.

STAGE 3 • AI SEARCH

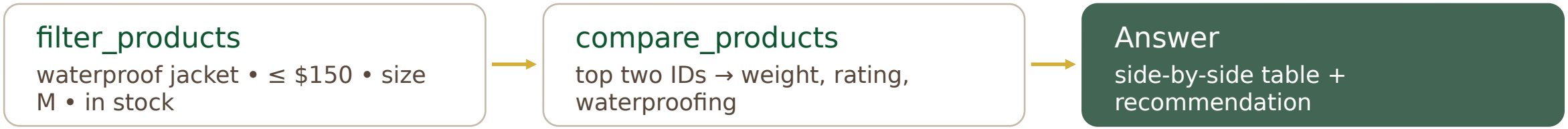
Our AI agent: it reasons, then uses tools

Built on Elastic Agent Builder. The LLM decides what to do; Elasticsearch does the retrieving.



Tool results go back to the LLM. It can call another tool, then writes a grounded answer with product cards and a comparison.

ONE SENTENCE, TWO TOOL CALLS



STAGE 3 • WHAT WE CONFIGURED

Three custom tools, three clear jobs

The LLM picks the tool and fills in the values. The tool decides how Elasticsearch is queried.

01

Product Discovery

`ecommerce.product_discovery`

TYPE Index search (ELSER)

JOB **Find candidates by meaning**

- Intent and paraphrases
- “jacket for a rainy commute”
- Broad needs like staying warm

02

Filter Products

`ecommerce.filter_products`

TYPE ES|QL, fixed query

JOB **Enforce exact rules**

- Price, size, stock
- Category and sort order
- Typed parameters only

03

Compare Products

`ecommerce.compare_products`

TYPE ES|QL, fixed query

JOB **Side-by-side specs**

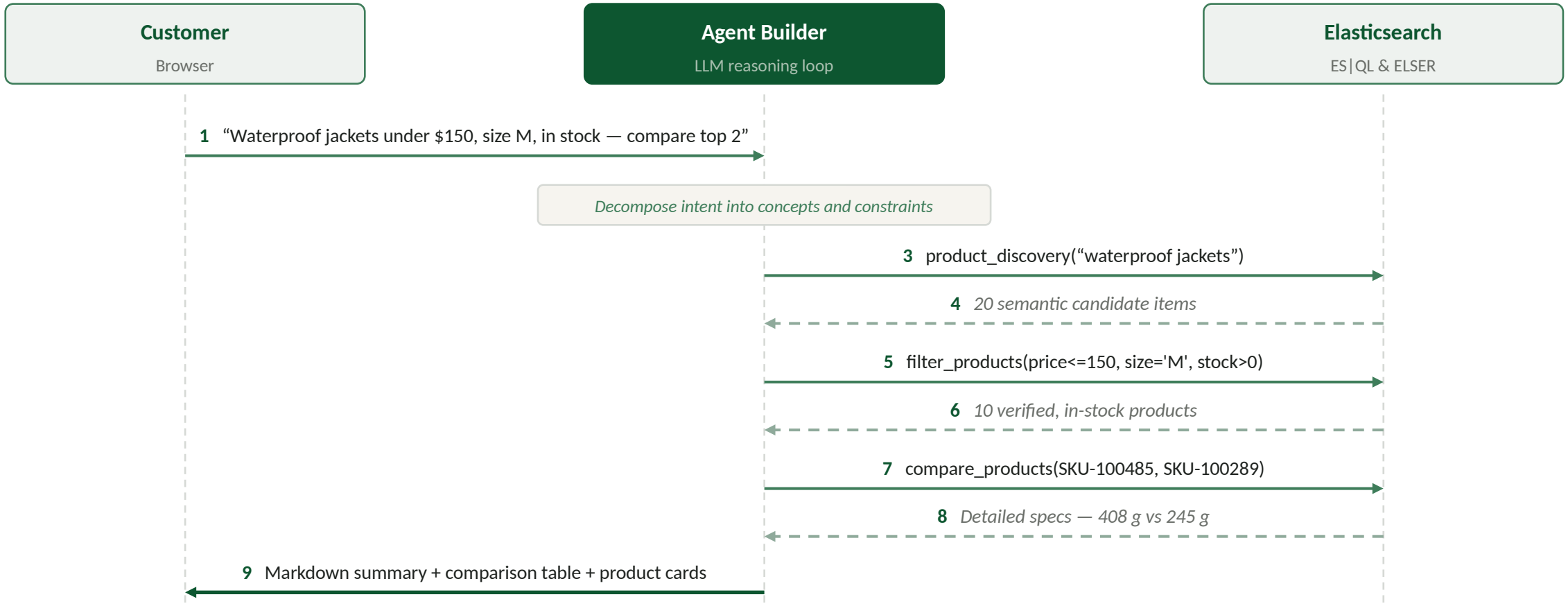
- Takes selected product IDs
- Weight, rating
- Waterproof rating

Guardrails: fixed ES|QL with typed inputs • scoped API keys • server-side validation of every answer

EXECUTION SEQUENCE

The multi-step agentic workflow in action

Sequence execution for a multi-constraint shopper request



Outcome: every claim in the final answer is backed by an authoritative catalogue row. **Zero hallucinations.**

WRAP-UP

Pick the right level for the job

Use the simplest approach that meets the need.

Classic search

Known words

- Product names, SKUs, brands
- Fastest and cheapest
- Fully explainable

ELSER

Meaning

- Synonyms and paraphrases
- Built into Elastic
- No model to train

AI agent

Action

- Exact rules + meaning together
- Compare and follow up
- Higher latency and cost

Better answers start with better evidence.

Questions?

